Qualité des données

Great Expectations

Bloc théorique — Jour 2 Après-midi

Michel Cadennes - mars 2026

Plan de la session

1. La qualité des données : un problème systémique
2. Les 6 dimensions de la qualité
3. Approches de validation : du SQL aux frameworks
4. Great Expectations — les 4 concepts fondamentaux
5. Stratégies de validation avancées
6. Intégration dans le pipeline DataOps
7. Limites, pièges et bonnes pratiques
8. Travaux pratiques — SciPulse

Le coût de la mauvaise qualité

3 100 Mds $

coût annuel pour l'économie

américaine (IBM, 2016)

Dashboard aberrant

Lignes dupliquées non détectées dans la source

Dégradation ML silencieuse

Distribution des données d'entraînement modifiée

Rapport resoumis

Format de date incorrect sur 2% des enregistrements

Migration corrompue

Contraintes d'intégrité non testées sur les données réelles

Caractéristique commune : détection en aval, longtemps après l'introduction du problème

Délai de détection : le facteur coût

Ingestion	Transformation	Mise à disposition	Rapport mensuel
Minutes	**Heures**	**Jours**	**Semaines**
Coût de correction			

← Plus tôt la détection, moins cher la correction →

Solution DataOps : le shift-left testing — valider le plus tôt possible dans le pipeline

Tester les données tester du code

01

Les données changent constamment

Un code ne change qu'au commit. Les données évoluent à chaque ingestion — nouvelles distributions, nouvelles valeurs. Un test peut échouer demain sans que le pipeline soit cassé.

02

Pas de spécification formelle

Le code implémente une user story ou un contrat d'API. Les données arrivent telles que la source les produit — schéma non documenté, valeurs nulles non prévues, conventions implicites.

03

Les problèmes sont souvent statistiques

Un bug logiciel est binaire (ça plante ou pas). Un problème de données peut être subtil : 0.3 % de valeurs nulles dans un champ qui n'en avait jamais eu. Est-ce un bug ou une évolution normale ?

Les six dimensions de la qualité

Complétude

Completeness

Toutes les données sont-elles là ?

Validité

Validity

Les valeurs respectent-elles les contraintes ?

Unicité

Uniqueness

Chaque enregistrement est-il unique ?

Cohérence

Consistency

Les données sont-elles cohérentes entre sources ?

Fraîcheur

Timeliness

Les données sont-elles suffisamment récentes ?

Exactitude

Accuracy

Les données reflètent-elles la réalité ?

Complétude et validité

Complétude

Niveau enregistrement : champs obligatoires remplis

Niveau jeu de données : tous les enregistrements attendus présents

Exemple SciPulse : un dump ArXiv habituellement à 100 000 articles qui passe à 50 000 — signal d'alerte.

Validité

Type : une date est bien une date

Format : l'ID ArXiv suit le pattern YYMM.NNNNN

Domaine : catégorie dans l'ensemble des catégories connues

Plage : date entre 1991 (création ArXiv) et aujourd'hui

Unicité

Pas de doublons — problème classique avec les pipelines qui relisent des fichiers (replay).

Dans SciPulse, l'arxiv_id doit être unique dans l'index.

Cohérence

Cohérence interne : primary_category ∈ categories[]

Cohérence inter-sources : si HN pointe vers un article ArXiv, cet article existe dans l'index.

Fraîcheur et exactitude

Fraîcheur (Timeliness)

Les données sont-elles assez récentes pour l'usage prévu ?

Se mesure par le delta entre le timestamp le plus récent et l'heure actuelle.

Ex. SciPulse : poller HN inactif depuis 24 h → données périmées.

Exactitude (Accuracy)

Les données reflètent-elles la réalité ?

La plus difficile à vérifier automatiquement — nécessite une vérité terrain.

Ex. SciPulse : comparer un échantillon d'articles avec leur page ArXiv.

Dimension	Exemple ArXiv	Expectation GX
Complétude	abstract jamais null	expect_column_values_to_not_be_null
Validité	Date entre 1991 et aujourd'hui	expect_column_values_to_be_between
Unicité	arxiv_id unique	expect_column_values_to_be_unique
Cohérence	primary_category ∈ categories	Script Python custom
Fraîcheur	Date max < 1 an	expect_column_max_to_be_between

Approches de la validation des données

1

SQL ad hoc

✔ Simple, aucune dépendance ✘ Non réutilisable, pas d'historique, pas de rapport → *Équivalent des print("debug") dans le code*

2

Tests de schéma (dbt, JSON Schema)

✔ Versionnés, déclaratifs, intégrés à dbt ✘ Limité aux contraintes simples (null, unique, valeurs) → *Bonne étape, mais pas suffisant seul*

3

Frameworks dédiés (Great Expectations, Soda, Deequ)

✔ 300+ validations, profiling, rapports visuels, historique, intégration Airflow ✘ Courbe d'apprentissage initiale → *L'ingénierie de la qualité au niveau industriel*

Great Expectations

Great Expectations : Philosophie

Les tests automatisés concernent aussi les données

Monde du code	Monde des données (GX)
Test unitaire	Expectation
Suite de tests	Expectation Suite
pytest run	Checkpoint
Rapport de couverture	Data Docs
assert x == 5	*expect_column_values_to_not_be_null("arxiv_id")*

« Great Expectations » — clin d'œil à Dickens : on déclare ses attentes, puis on vérifie si la réalité s'y conforme.

GX : Les concepts fondamentaux

Expectation

Une assertion sur les données. L'unité de base. GX en propose plus de 300 prédéfinies — sur les valeurs, les statistiques, la table entière.

Expectation Suite

Un ensemble d'expectations regroupées logiquement. L'équivalent d'un fichier de tests. Versionné dans Git comme le reste du code.

Checkpoint

Orchestre l'exécution d'une suite sur des données réelles. Le pytest run de la data. Peut déclencher des actions en cas d'échec.

Data Docs

Rapports HTML auto-générés : expectations définies, résultats des dernières exécutions, historique. Pilier de la collaboration DataOps.

Exemples d’attentes

Sur les valeurs d'une colonne

- expect_column_values_to_not_be_null
- expect_column_values_to_be_unique
- expect_column_values_to_be_between
- expect_column_values_to_match_regex
- expect_column_values_to_be_in_set

Sur les statistiques

- expect_column_mean_to_be_between
- expect_column_median_to_be_between
- expect_column_min_to_be_between
- expect_column_max_to_be_between
- expect_column_stdev_to_be_between

Sur la table entière

- expect_table_row_count_to_be_between
- expect_table_column_count_to_equal
- expect_table_columns_to_match_ordered_list

Le paramètre mostly

mostly=0.95 → au moins 95 % des valeurs doivent respecter la contrainte. Reconnaît l'imperfection des données réelles et évite les faux positifs permanents (alert fatigue).

Autres concepts principaux

2 — Expectation Suite

Ensemble d'expectations par source ou étape : arxiv_raw_suite · arxiv_enriched_suite · hn_raw_suite. Fichier JSON versionné dans Git.

3 — Checkpoint

1. Se connecte à la source
2. Exécute chaque expectation
3. Produit un rapport
4. Déclenche des actions si échec

Airflow : si le checkpoint échoue, la tâche est Failed → le pipeline s'arrête → les données corrompues ne se propagent pas.

4 — Data Docs

Rapports HTML auto-générés. Trois informations clés : quels tests existent · les tests passent-ils · depuis quand un test échoue-t-il.

Pilier de collaboration : n'importe qui ouvre le HTML et comprend l'état de santé des données sans lire le code.

Le processus Great Expectations en 4 étapes

1 CONNECT

Connecter GX à la source

(CSV, DataFrame Pandas, table SQL, Elasticsearch via export...)

2 EXPECT

Définir les expectations (déclarativement)

Approche manuelle ou assistée par le Data Assistant (profiling)

3 VALIDATE

Exécuter le Checkpoint

Chaque expectation est vérifiée — résultat success/failure par ligne

4 ACT

Consulter les Data Docs

Intégrer dans Airflow (stop si échec) · alerter si nécessaire

Sources supportées : CSV/JSON · DataFrame Pandas/Polars · PostgreSQL · Spark · (Elasticsearch via export Pandas)

Stratégies de validation

Brut vs. Transformé

À l'entrée — données brutes

On vérifie la FORME :

- Le fichier existe et est non-vide
- Les champs attendus sont présents
- Les types sont cohérents
- Le nombre de lignes est dans l'intervalle attendu
- Les identifiants sont uniques

Contrôles défensifs : on ne fait pas confiance à la source.

Après transformation — données enrichies

On vérifie le FOND :

- Les valeurs calculées sont dans des plages réalistes
- Les distributions sont cohérentes avec l'historique
- Les jointures ont produit le bon nombre de lignes
- Les enrichissements NLP ont fonctionné
- Les croisements HN / ArXiv sont cohérents

Contrôles sémantiques : on vérifie que le pipeline a bien fait son travail.

Tolérer l'imperfection

Exiger 100 % de conformité → faux positifs permanents → alert fatigue → les vrais problèmes passent inaperçus.

0.99 – 1.0 — **Champs critiques**

Identifiants, clés de jointure. Un seul manquant peut casser le pipeline.

0.95 – 0.99 — **Champs importants**

Dates, catégories. Quelques anomalies acceptables.

0.80 – 0.95 — **Champs optionnels**

Affiliations des auteurs, DOI. On sait que la source est incomplète.

< 0.80 — **Rarement utile**

Plus de 20 % de valeurs invalides → problème structurel à traiter autrement.

```
validator.expect_column_values_to_not_be_null("abstract", mostly=0.99)  # Critique
validator.expect_column_values_to_be_between("date_published", min_value="1991-01-01", mostly=0.95)
```

Le profiling comme point de départ

Pour un jeu de données inconnu, écrire des expectations à la main est fastidieux.

Data Assistant — profiling automatique

GX analyse chaque colonne (type, distribution, valeurs distinctes, nulls...) et génère un premier jeu d'expectations.

```
result = context.assistants.onboarding.run(batch_request=batch_request)

suite = result.get_expectation_suite()
```

1. Profiler les données avec le Data Assistant
2. Éditer manuellement : ajuster les seuils, supprimer les expectations non pertinentes
3. Ajouter les règles métier que le profiler ne peut pas deviner

Intégration

Intégration dans le pipeline DataOps — Airflow

download — **validate_raw** — ingest — transform — **validate_enriched** — serve

Si validate_raw échoue → ingest ne s'exécute pas. Si validate_enriched échoue → serve ne s'exécute pas.

```
download_arxiv >> validate_raw >> ingest_logstash >> transform_dbt >> enrich_python >> validate_enriched
```

Data Contracts

Le concept de data contract **formalise un accord entre producteur et consommateur de données.**

Dans SciPulse :

- Le producteur (dump ArXiv, API HN) doit fournir des données passant la suite raw
- Le pipeline (Logstash, Python, dbt) doit produire des données passant la suite enriched
- Le consommateur (Kibana, search_service) peut supposer que les données respectent le contrat

Data Docs : Documentation vivante

Au lieu d'un document Word décrivant le schéma (obsolète en deux semaines), les Data Docs montrent :

1. Quels tests existent sur chaque colonne **2. Les résultats de la dernière exécution (% de conformité)** **3. Les valeurs attendues (plages, ensembles, patterns)**

Un outil de collaboration puissant

Un data analyst qui découvre un jeu de données peut ouvrir les Data Docs et comprendre immédiatement les contraintes de qualité, sans lire le code du pipeline.

La documentation se met à jour automatiquement à chaque exécution du checkpoint.

Documentation statique

Word, Confluence → se périme dès le premier commit

vs

Data Docs (vivants)

HTML auto-généré → toujours à jour, toujours exact

Pour finir...

Limites et pièges courants

Abus de tests

- Trop d'expectations = faux positifs + maintenance coûteuse + bruit.
- Règle : tester uniquement ce qui casserait le pipeline en aval si c'était faux.
- Un abstract de 49 caractères au lieu de 50 : pas critique. Un arxiv_id null : toute la chaîne d'indexation casse.

Dérive des seuils

- Les données évoluent.
- Un seuil mostly=0.99 calibré en janvier peut devenir trop strict en mars si la source a changé de format.
- Révision régulière des seuils nécessaire — les Data Docs facilitent ce suivi en montrant les tendances.

La validation n'est pas la correction

- GX détecte les problèmes, il ne les corrige pas. Si un checkpoint échoue, un processus est nécessaire :
- 1. Diagnostiquer (source ? transformation ?) → 2. Corriger → 3. Relancer
- Le pipeline doit prévoir un chemin de correction

Travaux pratiques

1. Installation de Great Expectations et configuration du contexte
2. Rédaction de la suite ArXiv : complétude, validité, unicité sur les données brutes
3. Rédaction de la suite HN : validations adaptées à la structure des items
4. Exécution des checkpoints et consultation des Data Docs
5. Script de nettoyage Pandas guidé par les résultats : déduplication, normalisation des auteurs
6. Re-validation après nettoyage pour vérifier l'amélioration

Points clefs à retenir

Le **délai de détection** détermine le coût de correction — le shift-left est la réponse DataOps

6 dimensions de **qualité** : Complétude, Validité, Unicité, Cohérence, Fraîcheur, Exactitude

GX = 4 concepts :
Expectation · Suite · Checkpoint · Data Docs

mostly — calibrer les seuils selon la criticité, pas exiger 100 % (alert fatigue)

Dans Airflow : les **checkpoints** sont des barrages — zéro propagation de données corrompues

Les **Data Docs** : documentation vivante et automatique, pilier de la collaboration